

Towards a foundation model for partial differential equations: Multioperator learning and extrapolation

Jingmin Sun¹, Yuxuan Liu², Zecheng Zhang^{3,*} and Hayden Schaeffer^{2,†}

¹*Department of Mathematical Sciences, Carnegie Mellon University Pittsburgh, Pennsylvania 15213, USA*

²*Department of Mathematics, University of California Los Angeles Los Angeles, California 90095, USA*

³*Department of Mathematics, Florida State University Tallahassee, Florida 32304, USA*



(Received 12 October 2024; accepted 2 January 2025; published 12 March 2025)

Foundation models, such as large language models, have demonstrated success in addressing various language and image processing tasks. In this work, we introduce a multimodal foundation model for scientific problems, named PROSE-PDE. Our model, designed for bimodality to bimodality learning, is a multioperator learning approach which can predict future states of spatiotemporal systems while concurrently learning the underlying governing equations of the physical system. Specifically, we focus on multioperator learning by training distinct one-dimensional time-dependent nonlinear constant coefficient partial differential equations, with potential applications to many physical applications including physics, geology, and biology. More importantly, we provide three extrapolation studies to demonstrate that PROSE-PDE can generalize physical features through the robust training of multiple operators and that the proposed model can extrapolate to predict PDE solutions whose models or data were unseen during the training. Furthermore, we show through systematic numerical experiments that the utilization of the symbolic modality in our model effectively resolves the well-posedness problems with training multiple operators and thus enhances our model's predictive capabilities.

DOI: [10.1103/PhysRevE.111.035304](https://doi.org/10.1103/PhysRevE.111.035304)

I. INTRODUCTION

Partial differential equations (PDEs) are fundamental models for describing and understanding complex spatiotemporal processes in the physical and computational sciences. They provide one of the most important techniques for bridging experimental observations, physical principles, and mathematical properties. PDEs are effective in describing, analyzing, and predicting a wide array of real-world phenomena, including highly nonlinear, chaotic, and/or multiscale physics. Scientific computing (SC) problems concerning PDE center around simulating a specific equation or class of equations given a set of parameters, initial states, boundary conditions, etc. The objective is often to match experimental findings with formal models, forecast state variables, understand parametric dependencies, or obtain physical parameters.

When formulated as a machine learning task, solving PDEs amounts to approximating a mapping between input parametric functions and output solutions, equations, or more broadly, output information. In this work, we introduce the PROSE-PDE model, designed to be a foundation model for solving both forward and inverse PDE problems. The model is trained across various classes of time-dependent PDEs, including nonlinear diffusive, dispersive, conservation laws, wave equations, and others. The goal is to produce a neural network model that is able to generalize solutions based on different parametrized conditions and to extrapolate important physical phenomena between different governing systems.

This is a key step towards a foundation model for scientific applications.

Foundation models are deep learning models trained on large datasets often comprising of billions of learnable parameters in order to serve as base models readily applicable across a broad spectrum of applications [1]. They have significantly revolutionized natural language processing and the broader AI landscape through models such as BERT [2], GPT [3–5], DALL-E [6,7], stable diffusion [8], LLAMA [9,10], Claude, and others. Foundation models are general-purpose, allowing fine-tuning of parameters on additional datasets for new downstream tasks. However, to ensure their efficacy across a wide array of conditions, foundation models necessitate a significant amount of training data (trillions of tokens [9]) and computational resources.

While generative AI has seen unprecedented success in text-based tasks and image generation, its application to SC problems remains limited. There are several key differences that make SC problems challenging for current large language models (LLMs). First, accuracy and precision are crucial in SC applications. For example, a small numerical error in a coefficient of a (trained) governing equation can lead to catastrophic errors in downstream computing tasks, especially for chaotic, high-contrast, or multiscale systems where errors accumulate rapidly [11]. This also holds for other SC tasks such as forecasting state variables or optimizing structural parameters [12]. Second, SC problems typically yield unique solutions, e.g., there is only one solution to a well-posed PDE given a fixed set of conditions, whereas a sentence can convey a similar meaning with a variety of word choices. Additionally, while LLMs show some forms of reasoning abilities or emergent behaviors, they are not yet capable of numerical

*Contact author: zzhang14@fsu.edu

†Contact author: hayden@math.ucla.edu

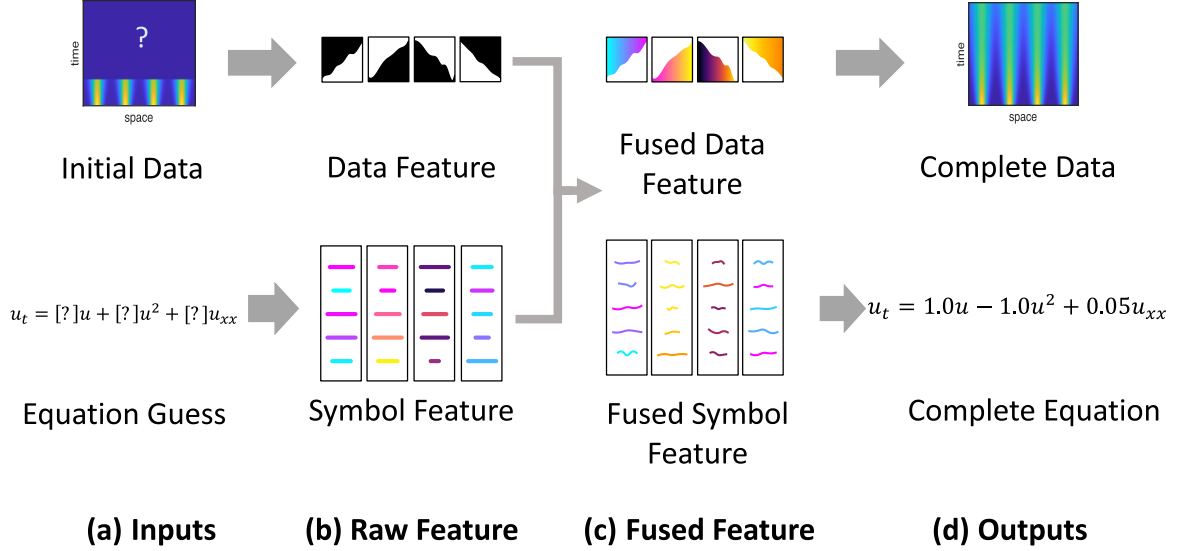


FIG. 1. PROSE-PDE workflow illustration: The inputs are the initial data (short-time observations) and a guess of the partial differential equation itself. The inputs are mapped into raw features (modality-specific) and then fused together. The fusion process couples cross-modality information. The decoders output a prediction of the data (as an operator) and write a complete mathematically valid equation.

or mathematical reasoning, including understanding relations, ordering, properties, and symmetries [13,14]. Although recent efforts have focused on chain-of-thought reasoning prompts to elucidate step-by-step reasoning processes, in Ref. [15] it was shown that a model could give a plausible argument that is consistent with the predicted answer but is an “unfaithful explanation of the model’s decision procedure.” For the trustworthiness of LLMs in the scientific domain, being able to provide reasoning or certified guarantees are necessary. Lastly, SC works in the data-scarce regime, as experimental data often requires time to acquire and is far less available compared with text or imaging databases. Thus, numerical simulations and heterogeneous data sources play a stronger role in training and testing foundation models for scientific applications.

A. Main contributions

This work introduces a multimodal neural network approach, PROSE-PDE, for predicting solutions of one-dimensional (1D) time-dependent PDE systems and for generating the underlying equations. The workflow of PROSE-PDE is illustrated in Fig. 1. The main contributions and novelties are summarized below.

(1) PROSE-PDE is the first multimodal transformer-based approach that encodes and decodes both numerical and symbolic datatypes (i.e., forward and inverse problems for multiple classes of PDEs). PROSE-PDE addresses the challenging problem of multioperator learning.

(2) We propose an approach that can generalize to new model or physical parameter values not encountered during training, to unseen timestamps or further into the future, to new initial condition distributions, to unseen physical systems, and to new physical features, all without fine-tuning. Three detailed studies, which include thirteen experimental settings, demonstrate the model’s extrapolation capabilities.

(3) We conduct two ablation experiments varying (1) the input length in time and (2) adjusting the weighting between the losses (data and symbolic), in order to examine the contribution of each of the two symbolic modalities (input and output) to the learning process. The results demonstrate the benefit of multimodal information and the model’s consistency under different training settings.

II. OVERVIEW

In this section, we introduce operator learning, provide an overview of the main technical aspects of the proposed model, and discuss related works.

A. Multioperator learning (MOL)

Let $G : U \rightarrow V$ be an operator, where U and V represent function spaces. Single-operator learning (SOL) aims to construct one neural operator G_θ parametrized by θ to approximate the operator G . In Refs. [16,17], the authors proved the first universal approximation theorem for nonlinear operators, which has led to several recent approaches in operating learning. For example, the popular deep operator neural network (DON) [18–22] and its variants [23–25] approximate the operator by learning the function bases and have the benefit of not requiring a fixed discretization for the output function. Fourier neural network (FNO) [26–29] utilizes Fourier layers within the network architecture and thus is invariant to the input discretization. These advancements have led to successes in applying neural operators to various real-life engineering challenges; for example, those in climate [30,31], earth structural [32], physics and astronomy [33–35], biology [36], power systems [22], optimization and UQ [21,37], multifidelity modeling [38–41], etc.

SOL has been used in solving mathematical and scientific computing problems. For example, SOL can construct the solution operator of PDEs and learn the mapping from the

initial condition of the system to the solution. Specifically, let us denote a parametrized PDE system $\mathcal{S}$ as

$$\begin{aligned}\mathcal{L}(u(x, t; q)) &= 0, & x \in \Omega, \\ \mathcal{B}(u(x, t; q)) &= 0, & x \in \partial\Omega, \\ u(x, 0; q) &= \mathcal{G}(x; q), & x \in \Omega, \quad q \sim \mathcal{D},\end{aligned}$$

where $\mathcal{L}$ and $\mathcal{B}$ denote the governing equations and boundary conditions. The initial condition $\mathcal{G}$ is a generating function, q denotes the parameters that determine the initial conditions, and $\mathcal{D}$ is the distribution. Thus, one task in SOL is to train the mapping from $\mathcal{G}(x; q)$ to $u(x, t; q)$, i.e., learn $G \approx G_\theta$.

A central goal in scientific machine learning is to develop methods that are able to extrapolate, i.e., to solve tasks beyond those encountered during training. While a trained SOL model may effectively address a specific operator and task, it could encounter difficulties when presented with new tasks. For instance, if G_θ learns an approximation to the solution operator $G^{(0)}$ for system $\mathcal{S}^{(0)}$, it may struggle to handle a new task represented by a different operator $G^{(1)}$ associated with a new system $\mathcal{S}^{(1)}$. Thus to address this challenge, we develop a multioperator learning (MOL) approach, which entails training a single foundation model to learn multiple operators. Specifically, the objective is to establish an umbrella mapping that can map many distinct encoded operators (with their corresponding input functions) to an accurate approximation of their output function.

Several MOL methods [42–49] have recently been proposed. Most of these methods provide a label (either implicitly or explicitly) to the inputs, signaling to the model which among the many operators to use. The label is one approach for dealing with the issue of well-posedness, i.e., disambiguating different equations with similar initial values. The PROSE (predicting operators and symbolic expression) approach [42] is a multimodal foundation model capable of constructing operators while simultaneously learning equations. Utilizing trainable symbols to encode operators, PROSE has demonstrated efficacy in encoding high-dimensional dynamical systems and for learning mathematical representation of the data.

The in-context operator network (ICON) [43–45] uses an in-context learning approach for MOL, where existing data from the encountered system is used as implicit labels for the model. In Ref. [46], the model uses a graph transformer to process the graph representation of the equation, which may suffer from long-range dependency issues and is limited to forward problems. In Ref. [47], a pretrained LLM is used to generate equation labels for the model, and the model’s ability to reason about equations is thus dependent on how well the LLM understands mathematical and numerical values. Multiple physics pretraining (MPP) [48] directly learns from only the history of the system. Since no labels are used, MPP may suffer from the well-posedness issues.

While many of these methods show some ability to generalize to new system parameters outside of the training range; none have demonstrated an ability to generalize to unseen systems and extrapolate physical phenomena without the use of fine-tuning. As the training dataset is rarely exhaustive, this is a natural requirement for SC foundation models. In this work, we propose a multimodal scientific foundation model,

PROSE-PDE, in the context of 1D PDE problems and we show the model’s extrapolation capabilities.

B. Multimodal machine learning

Multimodal machine learning (MMML) trains models using data from heterogeneous sources [50–56] and solves multiple tasks simultaneously. For example, for visual-language reasoning [51,52,57], models utilize visual content, i.e., images or videos, combined with the semantics of language [52] associated with these visual elements, such as captions or descriptions. This has led to the development of models with richer information [57]. MOL tasks inherently belong to the realm of MMML tasks, specifically characterized as bimodal tasks involving functions and operator encoding as two heterogeneous inputs.

The PROSE-PDE model takes a further step: it also generates a governing equation of the system, making it a bimodal input bimodal output model. We present the details in Sec. III. Numerous innovations contribute to the efficacy of large-scale foundational multimodal models, with one pivotal advancement being the integration of attention and transformer structures. The attention mechanism allows for complex sequence encodings, adept at capturing sequential dependencies within data [58–60]. When processing features, this mechanism evaluates the significance of various segments within the sequence, generating distinct encodings for each segment. It then utilizes these encodings to attend to different parts of the sequence by varying the weights, fostering intrasequence connectivity. The classic transformer architecture leverages self-attention [61,62], enabling it to capture intricate relationships within lengthy sequential data. Conversely, cross attention enables the model to discern connections between distinct sequences, thereby facilitating the learning of intermodality relationships. More details are included in Appendix C.

C. Extrapolation of physical features (EPF)

Abstraction is a fundamental aspect of the scientific method, in particular, the pursuit of a deeper understanding of the underlying mechanisms behind observed phenomena. In this work, we validate our approach by assessing its ability to generalize across different input conditions and its capabilities of extrapolation of physical features, which we refer to as EPF. In the setting of foundation models for scientific domain problems, extrapolation entails evaluating whether the model has learned the underlying physical laws to a sufficient extent to generalize to either new physical systems or new conditions, possibly through a transfer of fundamental rules or key features. While there is no clear definition to the extent one expects a neural network to extrapolate spatiotemporal systems, some useful capabilities include

- (1) generalizing to new model or physical parameter values not encountered during training.
- (2) predicting variables at unseen timestamps or further into the future (forecasting).
- (3) handling new condition classes, such as changes in the smoothness of the initial state or the form of the parametric input functions.

(4) generalizing to new physical systems not seen in training.

In addition, a scientific foundation model should have the EPF property, since physical phenomena are often shared across systems that have similar underlying laws. For this work, we focus on the generalization of physical features in conservative systems, specifically, we show that our model has the EPF property by

(1) training the model on a dataset containing simulations with only single observed shocks, while testing it on settings with multiple shocks (extrapolating shock interactions).

(2) varying the training sets with mixtures of shocks and rarefactions (with differing amounts per physical system) and asking the model to predict a shock or rarefaction on a new system (transferring physical laws to a new system).

All of these EPF tests are challenging for classical and current techniques; however, they are essential for demonstrating progress toward developing a general-purpose large-scale model for physical systems. We focus on smaller-scale problems to show that these properties hold even with (1) limited data or (2) a smaller amount of trainable parameters as compared with standard LLMs. Notably, these tests provide insights into PROSE-PDE's capacity for abstraction or can at least measure its potential to extract underlying rules from PDE data.

III. METHODOLOGY

The main components of the PROSE-PDE architecture include transformers, symbolic encoding, and multimodal inputs and outputs. We summarize some key elements in this section and provide an overview of the PROSE-PDE architecture and workflow. More architecture details are in Appendix B.

A. Equation encoding via Polish notation

A central challenge in MOL is encoding operators so that the encoded representations can seamlessly adapt to new, unseen operators. We employ symbolic encoding to address this challenge. The symbolic encoding of mathematical operators has been studied in Ref. [42] and it is also used in other mathematical problems [63,64]. However, its extrapolation abilities to new operators have undergone limited investigation. Through numerical experiments in Sec. V A, we demonstrate that the use of symbolic encoding facilitates extrapolation. We illustrate the symbolic encoding of an equation in Fig. 2. This encoding involves representing the equation in a tree structure, where nodes represent operations and leaves represent variables and constants. We then convert the tree structure into Polish notation [65]. Throughout the training process, all symbols in the Polish notation are considered trainable tokens and are updated accordingly. For further details, please refer to Refs. [42,66–70].

B. Model overview

The PROSE-PDE architecture contains five main components: data encoder, symbol encoder, feature fusion, data decoder, and symbol decoder. All of these components use variations of the attention structure. In this section, we present

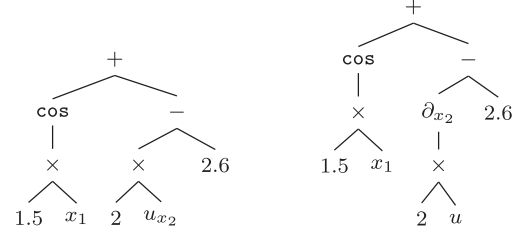


FIG. 2. Two equivalent tree encodings of the example expression $\cos(1.5x_1) + 2u_{x_2} - 2.6$. The left tree directly uses the partial derivative symbol u_{x_2} , while the right tree uses a differential operator symbol ∂_{x_2} . We adopt the left approach for the tests in this work

the workflow of PROSE-PDE and illustrate how information is processed in the network (see Fig. 3). Technical details of each component are discussed in Appendix B.

There are two types of inputs to the PROSE-PDE network: the initial data sequence and the symbolic equation guesses. The two input modalities are first transformed into sequences of feature vectors, making them more suitable for subsequent tasks. A small feedforward network independently upsamples each element in the input data sequence, generating a sequence of high-dimensional feature vectors. The sequence is then processed by the data encoder, allowing information to flow across elements in the sequence (e.g., the data encoder can locate the minimum and maximum, find peaks and modes, and detect patterns in the sequence).

The symbolic equation guesses are transformed into a sequence of symbol tokens, which is processed in the same way as a sentence in language tasks. That is, the tokens are first independently transformed into trainable feature vectors and then processed using a transformer structure in the symbol encoder. Similar to the data encoder, the transformer structure allows for information exchange across elements in the sequence and for the construction and interpretation of mathematical structures from the symbolic guess inputs.

The processed data and symbol sequences are then concatenated and fed into the feature fusion block, where modality interaction and fusion occur. The data features obtain information from the symbolic input (e.g., aspects of the equation underlying the data), and the symbolic features are refined using the data (e.g., rough parameter ranges based on the data provided). After the information exchange, the fused features are ready to be decoded into corresponding outputs.

The data decoder constructs the operator by synthesizing two independent input sources. One source is the fused features from the feature fusion block, which can be interpreted as basis functions for the output space. The other source is the query time points, which are separate from the main information flow of the network. Together, the data decoder learns to evaluate the output basis functions at locations specified by the query time points and combines them to generate the output predictions.

The symbol decoder is a standard encoder-decoder transformer for sentence generation, where the fused features act as the context guiding the output expression generation. The symbol decoder autoregressively generates the output sentence [58,71] from scratch, until it encounters the end-of-sentence signal. As the output sentence is starting from

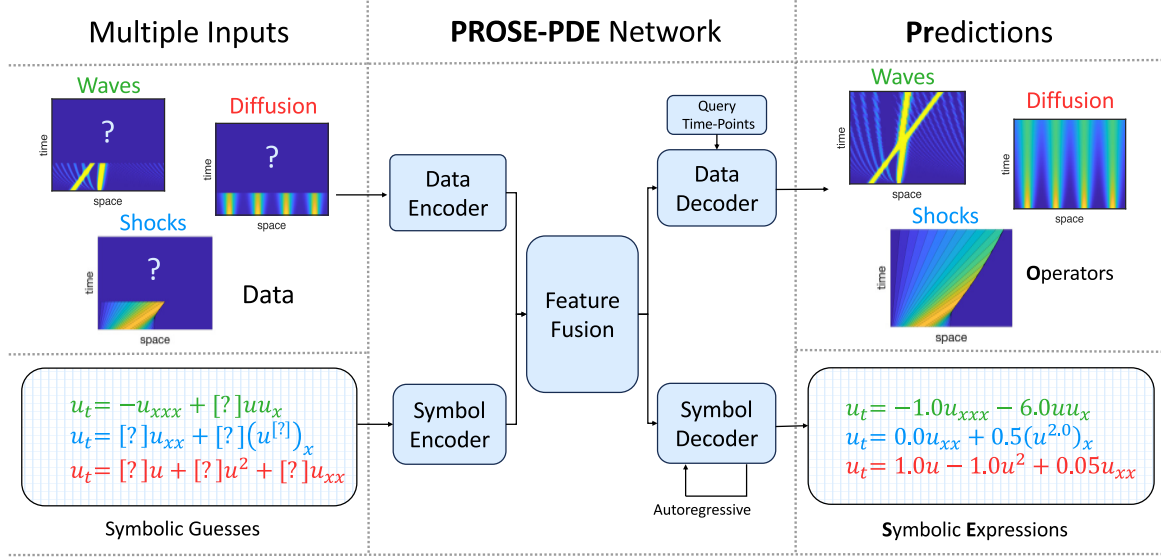


FIG. 3. PROSE-PDE network and the workflow. Data input and symbolic guess input are transformed into feature vectors, which are then processed by data and symbol encoders. The processed feature vectors are combined through the feature fusion block to allow information exchange and interaction. The resulting fused features contain information from both sources and are inputs to the output structures. The upper-right data decoder structure constructs the output operator based on fused features, where a separate set of query time points serve as evaluation points. PROSE-PDE generates symbolic expressions in the lower-right portion autoregressively.

scratch, the model has the ability for self-correction and refinement. More specifically, it can simultaneously remove incorrect terms, generate missing terms, and identify unknown coefficients, without the need for postprocessing or task-specific modules.

IV. EXPERIMENTAL SETUP

We study a diverse set of 20 PDEs with distinct physical features. To expand our study, we create a family for each PDE by randomly sampling PDEs' parameters from a uniform distribution within the range of $[0.9q_c, 1.1q_c]$, where q_c is the point-of-interest with an additional $\pm 10\%$ value variation. This process generates a dataset containing 10.24 K PDEs. To provide a closure of the PDEs, we equip each PDE with 50 initial conditions. Consequently, we create and investigate a dataset with $10.24 \text{ K} \times 50 = 512 \text{ K}$ systems in total. We refer to Appendix A for details.

The output of the model is a prediction of the solution at (future) time points given some short-time observations and, simultaneously, a refinement of the equation guess. The input function is sampled at 16 timestamps from $[0, t_f/2]$ and on a spatial grid of 128 points in $[0, x_f]$. The final time t_f may vary between different PDE, but the spatial grid length is set to $x_f = 2$ for all systems except for the Fokker-Planck equation. We rescale and normalize all the equations via change of variables to share the same t_f and x_f for the same experiment. The target operator maps the inputs to the solution at timestamps in the latter half of the interval $t \in [t_f/2, t_f]$ with the same spatial grid.

A. Evaluation metrics

We use four evaluation metrics to assess the behavior of PROSE-PDE in data prediction and symbolic learning. For

measuring the error for the data, the relative L^2 error and the R^2 score [$R^2 = 1 - \sum_i \|\mathbf{v}_i - \mathbf{u}_i\|_2^2 / \sum_i \|\mathbf{v}_i - \text{mean}(\mathbf{v}_i)\|_2^2$] are used, where $\mathbf{u}_i$ is the i th predicted solution and $\mathbf{v}_i$ is the i th target solution.

For the symbolic expression output, we first decode the symbolic Polish notation into trees representing functions, then the percentage of decoded output that can be transferred to valid mathematical expressions is reported. The valid expressions (which are differential operators) are applied to a set of functions randomly generated from the span of some basis set and then evaluated on a uniform space-time grid. Common choices of basis functions are polynomials and trigonometric functions [72–75]. We use tensorized polynomials with degree four in space and degree two in time. More precisely, suppose $f(\cdot)$ and $\hat{f}(\cdot)$ are the target and PROSE-PDE generated solutions, the relative L^2 error, $\|f(P) - \hat{f}(P)\|_2 / \|\hat{f}(P)\|_2$ is reported, where P is randomly generated. In particular, we define $P(x, t) := P_1(x)P_2(t)$, where $P_1(\cdot)$ is a degree four polynomial, $P_2(\cdot)$ is quadratic, and all the coefficients are randomly sampled from the uniform distributions on $[-5, 5]$. The relative L^2 error is approximated on a uniform 128×64 grid in the region $x \in [0, 2]$ and $t \in [0, 2]$.

B. PROSE-PDE modality configurations

We explore three different modality configurations for PROSE-PDE across various tasks. The first configuration, known as the two-to-two model, incorporates all five structures: data encoder, symbol encoder, feature fusion, data decoder, and symbol decoder, as illustrated in Fig. 3, and thus learns the solution operator and predicts the equations. This model demonstrates PROSE-PDE's capability to accurately reconstruct symbolic expressions. The second configuration, the two-to-one model, omits the symbol decoder and utilizes

TABLE I. Experiment settings and results. Data-noise: 2% additive noise on data. For the known case, the equation input is known thus the relative symbol expression error measures the ability to correctly encode and regenerate the symbol space. For the skeleton case, the symbolic inputs have unknown numerical values in the equation.

Expression	Data-noise	Unknown coefficients	Relative (data) prediction error (%)	R^2 score	Relative symbol expression error (%)	Valid fraction
Known	✓	✗	0.92	0.998	0.01	100.00%
Skeleton	✓	✓	1.06	0.998	0.768	99.94%

the remaining four components of PROSE-PDE. The two-to-one model is used to investigate PROSE-PDE’s ability to generalize to unseen operators with complete equations encoded as trainable symbols.

The final configuration, referred to as the one-to-one model, only uses the data encoder and data decoder. This model serves as a reference in assessing the significance of the symbolic component in enhancing our understanding of the operator and used in the ablation tests. For the sake of clarity, we also refer to operator input-output (I-O) as data I-O and symbolic expression I-O as symbol I-O.

V. RESULTS

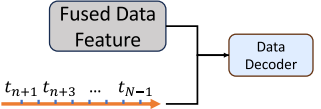
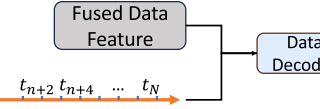
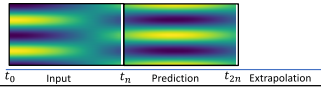
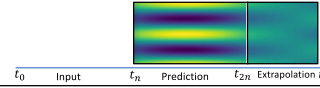
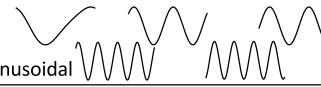
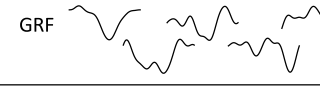
We first evaluate the performance of PROSE-PDE by assessing two two-to-two model settings: the “known” and the “skeleton” cases. For the “known” case, we provide the network with complete knowledge of the input equation. Thus, we expect that the network uses the encoded equation (symbol part) as an identifier and the primary evaluation is on predicting the output data. In the “skeleton” case, the symbolic input to the network is the equation with the coefficients replaced by a placeholder, i.e., the numerical values in the equation are

unknown. The objective is to simultaneously construct the data and learn the equation. We observe in Table I that in both cases, a low relative (data) prediction error ($<1.06\%$) and high R^2 score is achieved. The skeleton case recovers the unknown equation with a low error (0.768%). Detailed results for each operator are presented in Appendix E.

A. Extrapolation studies

Classical numerical methods have reliable and highly accurate performance on many SC tasks, e.g., solving the initial boundary-value problem. Machine learning (ML) methods on the other hand have superior performance in improving repeated computations and for utilizing large amounts of data. However, the reliance on training data leaves ML’s scientific extrapolating abilities open. The concept of extrapolation has various interpretations across scientific disciplines. In computer vision, for image classification tasks it takes the form of classifying an image belonging to a class the model has not seen before. To achieve this, a model is trained on a dataset containing N classes, where each class has K samples. If we then present the model with a new dataset containing M completely different classes, and the model

TABLE II. Study 1: Various extrapolation results. *Temporal grid* (basic setting for all experiments): Different query points (independent variables of the output functions) in training and testing. *Time marching*: Predict further steps from training. *Out-of-distribution*: Disjoint range of free coefficients in testing. *Input function class*: Periodic initial condition in training, 1D Gaussian random field (GRF) in testing. *Unseen operators*: Test on an unseen equation.

Extrapolation Settings	Training Dataset	Testing Dataset	Testing Metrics	
			Rel- L^2	R^2
Temporal Grid			0.92%	0.998
Time Marching			5.22%	0.893
Out-of-Distribution	$q_i \in (0.9 q_{i,c}, 1.1 q_{i,c})$	$q_i \in (0.8 q_{i,c}, 0.9 q_{i,c}) \cup (1.1 q_{i,c}, 1.2 q_{i,c})$	2.36%	0.986
Input Function Class	 Sinusoidal	 GRF	2.78%	0.987
Unseen Operators	$u_t + q_1(u^3)_x = q_2 u_{xx}$ $u_t + q_1 \sin(u)_x = q_2 u_{xx}$ $u_t + q_1(u^3)_x = 0$ $u_t + q_1 \sin(u)_x = 0$ $u_t + q_1(u^2)_x = 0$	Unseen during training: $u_t + q_1(u^2)_x = q_2 u_{xx}$	5.41%	0.945

successfully classifies an image from this new dataset into one of those classes, this is considered a zero-shot extrapolation task.

We formulate several extrapolation settings related to SC tasks and examine the ability of PROSE-PDE (two-to-one) model to extrapolate in these settings. We first demonstrate the robustness of our model through four different types of extrapolation presented in Table II. We focus on the data prediction capabilities so the network is given full equation information. The following are the four settings used in Study 1.

Temporal grid. During the testing phase, the trained model should be able to predict the output function at different points t in the domain, that is $t \neq t_k$, where t_k is from the training set. This is the standard setting used in validating all our numerical experiments (including Table I). This is also a measure of extrapolation since t is sampled outside of the training domain.

Time marching. During the training stage, the model takes data input from $t_{\text{in}} \in [0, 1]$ and predicts the solution in $t_{\text{pred}} \in [1, 2]$. During testing, we roll out the model to obtain solutions for longer intervals as follows. After obtaining predictions for times $[1, 2]$, we use them as the input and repeat the predictions for $t_{\text{pred}} \in [2, t_{\text{end}}]$. We present the results for $t_{\text{end}} = 2.25$ in Table II. As expected, the L^2 error increases as we increase t_{end} : when $t_{\text{end}} = 2.5$, $E_{\text{pred}} = 7.09\%$, and when $t_{\text{end}} = 3$, $E_{\text{pred}} = 10.09\%$, where E_{pred} stands for the relative L^2 prediction error.

Out-of-distribution (OoD). We study the model's ability to generalize beyond the training distribution. During testing, we sample $q \sim \mathcal{D}'$, where $\mathcal{D}'$ represents a distribution larger than the training parameter distribution $\mathcal{D}$. Specifically, we choose the random coefficients by sampling $q \sim \text{Uniform}(\lambda_1 q_c, \lambda_2 q_c)$, where q_c are the points of interest. The values for (λ_1, λ_2) are chosen to be $(0.9, 1.1)$ in the training phase, whereas $(\lambda_1, \lambda_2) = (0.8, 0.9), (1.1, 1.2)$ are used during testing. This shows the model can be used to predict solutions even if the coefficients are not seen in training.

Input function class. We extend the concept of OoD by changing the generating function, i.e., testing samples are produced using a distinct generation function $\mathcal{G}'$ along with a different parameter distribution $\mathcal{D}'$. For Table II, in training we used periodic initial conditions generated as sums of sinusoidal functions, whereas in testing, the initial conditions are generated using Gaussian random fields (GRF). The testing case has larger variations and is thus less regular than the training set. We refer to Appendix A and Table VI for details.

Unseen operators. We aim to assess whether the trained neural operator can adapt to operators unseen during training. Specifically, we test the model with operators G' that are not included in the training operator set $\{G_i\}_{i=1}^{N_o}$. We use five operator families $\{G_i\}_{i=1}^5$ in training, and evaluate the operator on the viscous Burgers' equation G' in testing. We randomly sample the free parameters of the PDEs and generate 128 K systems for training and 102.4 K systems for testing. These results show the capability of PROSE-PDE to learn a new operator G' without any fine-tuning.

Extrapolation of physical features studies

In this section, we demonstrate the EPF property of the proposed network. We show that PROSE-PDE can transfer unknown physics features by learning similarities from other

operators and predict such physical features even in the absence of exposure during the training phase.

In the input function class extrapolation tests, changes in the initial conditions can lead to new phenomena in the outputs. However, this directly depends on the equation and the input function class. As an example, consider the case where the training set is generated by randomizing the (finite) Fourier series and where we test on the Riemann problem, e.g., the initial data in the testing phase is generated by a step function:

$$f(x) = \begin{cases} 0 & \text{if } x \in [-\pi, 0] \\ 1 & \text{if } x \in [0, \pi]. \end{cases}$$

Then, although this initial condition was not seen during training, it could be approximated by, for example, $h(x) = \frac{1}{2} + \frac{2}{\pi} \sin(x) + \frac{2}{3\pi} \sin(3x)$, which may be seen in the training phase. While still an extrapolation test, the testing samples may bear some resemblance to the training samples. In Studies 2 and 3, we propose a more demanding extrapolation test. Specifically, we pick the testing generator such that the output functions exhibit fundamentally different physical characteristics.

Study 2: Transferring physical features. The two prevalent physical phenomena that are observed in conservation laws are shocks and rarefaction waves. The objective of Study 2 is to investigate PROSE-PDE's ability to generalize these physical phenomena between distinct equations. To test this, we design a series of experiments in which we change the proportion of shocks and rarefaction waves sampled in training data and measure the model's ability to predict unseen rarefaction waves in testing.

The data are generated using the Riemann problem with initial conditions located within the interval $[0, 1]$, and with homogeneous Neumann boundary conditions. All testing is done on the rarefaction setting for the viscous Burgers' equation, which is not used in training. The shock setting for the viscous Burgers' equation is included in the training set to help link it to other conservation laws, i.e., help with the transferring process. For instance, in Experiment 1 (first row of Table III) the training dataset includes rarefaction waves from all equations except the target equation and shocks only from the target equation. Conversely, in Experiment 5 (last row of Table III), the only equation in the training dataset exhibiting a rarefaction solution is the cosine flux equation. Note that each row of Table III includes about 3 K randomly generated systems in the listed types for a total of 153.6 K systems for training and 20.48 K random systems for testing.

We observe from the Table III that PROSE-PDE can construct rarefaction waves for viscous Burgers' systems without directly observing them. This shows the model's generalization and EPF capabilities. Specifically, PROSE-PDE likely learns the mechanism behind the rarefaction wave based on the training data of other systems and generalizes it to the viscous Burgers' setting.

Remark 1. To check if the network memorizes rarefaction features from other equations and applies those directly to the target equation during testing, we measure the similarity of the training systems as follows: We generate the solutions for the target equation and the solutions of the other randomly sampled systems by reusing the same initial conditions. We

TABLE III. Study 2: Transferring physical features. Each row in the table represents a distinct experiment. The purple region indicates that the training data corresponding to the listed PDE type (the columns) consists of shock solutions, while the blue region indicates that the training data corresponding to the listed PDE type consists of rarefaction waves. As a reference, in Experiment 5 the prediction error 2.34% is lower than directly using a fitted cosine flux as a prediction, which yields 3.59% error.

Exp No.	Training Sample						Testing Sample	Rel- L^2 Error
	Viscous Burgers'	Burgers'	Viscous Cubic Flux	Cubic Flux	Viscous Cosine Flux	Cosine Flux	Viscous Burgers'	
Exp 1								0.49%
Exp 2								0.54%
Exp 3								0.49%
Exp 4								0.55%
Exp 5								2.34%

define the similarity e_i by

$$e_i = \|G_{\text{target}}[u](\cdot) - G_i[u](\cdot)\|_2 / \|G_{\text{target}}[u](\cdot)\|_2, \quad (1)$$

where u is the initial condition (leading to rarefaction waves), G_{target} is the true viscous Burgers' solution operator, and G_i is the true solution operator of the i th training equations.

The computed similarities over the training equations are 1.38% for Burgers', 14.16% for viscous cubic flux, 13.66% for cubic flux, 1.85% for viscous cosine flux, and 3.59% for cosine flux, respectively. Note that the prediction errors for each experiment in Table III are consistently lower than the minimum of these values over the corresponding experiment. This suggests that the PROSE-PDE model extrapolates the data rather than merely replicating the best (training) operator.

Study 3: Generalizing to multiple shocks. We examine the model's ability to generalize single shocks to multiple shock interactions, specifically concentrating on four PDE types all in polynomial flux forms. In particular, PROSE-PDE is trained using data that leads to one shock (purple region of Table IV) or multiple shocks (blue region of Table IV). The initial conditions are set within the range $[0,1]$, with homogeneous Neumann boundary conditions.

Table IV details each experiment setting and demonstrates the model's ability to accurately resolve the behavior of two shocks in the viscous cubic flux setting, even when the exact physical feature is not present during training. We verify that the network is not memorizing the training data by checking the similarities as defined in Eq. (1), with G_{target} representing the true viscous cubic flux solution operator. The recorded similarities are 1.15% for cubic flux, 16.06% for viscous Burgers', and 15.74% for Burgers' equation. The prediction errors shown in Table IV remain consistently below these similarity values, indicating that PROSE-PDE effectively extrapolates to two shock interactions for the target operator, rather than simply repeating what is observed in the training dataset.

B. Ablation studies

The PROSE-PDE model is a bimodal to bimodal model, specifically, it maps the data and symbols to predicted data and symbols. In this section, we show that both symbolic input and output modalities enhance the learning of data prediction.

1. Ablation study: Symbolic encoder-decoder

In PROSE-PDE, trainable tokens are used to represent the input equation and thus help to identify the operator. In this section, our objective is to illustrate that such representation can effectively discern between different operators. We conduct a comparison between the PROSE-PDE two-to-one models, the model utilizing only the data modality (one-to-one model), and two single-operator learning models (FNO and DeepONet). Note that the input functions are only sampled at the initial timestamp, thus the learning problem is ill-posed for a model relying solely on the data modality. Since the two-to-one model leverages the equation information, we expect the learning to remain well-posed. As shown in Table V, PROSE-PDE two-to-one (first row) produces reliable predictions with small errors (1.03%) even when supplied with only the initial conditions as data input, showing that benefit of the symbol encoder.

To test the model's dependency on the number of timestamps for the input function, we test the PROSE-PDE two-to-two model with skeleton inputs and vary the number of timestamps, i.e., the input size. Figure 4 shows the consistent behavior of our model over the input size, which indicates the stability induced by the symbol encoder-decoder in providing additional information for the data prediction.

2. Ablation study: Loss weights

Lastly, we want to demonstrate the robustness of the model with respect to the importance of minimizing the loss for each output modality. This is done by varying the weights between

TABLE IV. Study 3: Shock interactions. Each row in the table represents a distinct experiment. For each listed PDE type (the columns), a purple region indicates that the training data of this PDE type are single-shock solutions, while a blue region indicates that the training data of this PDE type are multishock solutions. As a reference, in Experiment 3 the prediction error 2.35% is lower than directly using the Burgers' equation with the same initial conditions, which yields 16.06% error.

Exp No.	Training Sample				Testing Sample	Rel- L^2 Error
	Viscous Cubic Flux	Cubic Flux	Viscous Burgers'	Burgers'	Viscous Cubic Flux	
Exp 1						0.77%
Exp 2			Two Shocks			1.02%
Exp 3		One Shock				2.35%
Exp 4						4.48%

the two output losses. Specifically, in Fig. 5, we change the weights assigned to the data and symbol losses. We observe a slight increase in the error for the data output when we reduce the weight ratio (data weight over symbol weight) from 5 to 0.2. However, the overall data-prediction error remains low. This suggests that the output symbol modality contributes to the overall improvement through the fusion layers. That is, by including some symbolic information, the model's data prediction remains resilient to changes in the training hyperparameters.

To illustrate this, Fig. 6 shows the gradient information flow for the PROSE-PDE model during the training phase. Assigning a larger weight to the symbol component enhances the learning of the lower portion of the PROSE-PDE model. However, this adjustment negatively impacts the data decoder for data learning. We hypothesize that the inclusion of the symbol modality enhances the learning of the fusion structure, thereby generating a stronger encoding for the operator (data prediction). Consequently, we only observe a slight decrease

in performance for data prediction which underscores the advantages of the symbol modality.

VI. DISCUSSION

The PROSE-PDE approach is a bimodality to bimodality model for solving forward and inverse tasks in multioperator learning of spatiotemporal systems. The main focus of this work is on presenting and testing an architecture for a PDE foundation model for time-dependent nonlinear partial differential equations that can generalize. Through detailed experiments, the PROSE-PDE approach is shown to generalize in various settings without the need of fine-tuning. Most importantly, the model is able to extrapolate some physical features as verified by testing the extrapolation capabilities for rarefactions and multishock interaction. We expect stronger capabilities with a larger training set. The ablation results demonstrated the importance of multimodal information for generating consistent and robust results. A future direction is

TABLE V. Ablation study: Comparing the two-to-one model to data-only architecture and two single-operator learning networks (FNO and DeepONet) using the data-prediction error. Note that the input functions are only sampled at initial time step to predict all future states. The error are indicated as extrapolation in temporal grid, input function class, or out-of-distribution. The errors are indicated as extrapolation in temporal grid, input function class, and out-of-distribution as described in Sec. V A. The results indicate the need for symbolic information to discern operators in MOL.

Modality	Testing metrics for data prediction	
	Rel- L^2 error (%)	R^2 score
Data/symbol encoder + fusion + data decoder (two-to-one)	1.03/3.03/3.52	0.997/0.982/0.972
Data encoder + data decoder (one-to-one)	29.5/41.21/30.86	-1.095/-0.56/-0.99
FNO	34.86/37.32/35.42	-2.82/-1.82/-2.28
DeepONet	32.07/37.88/32.82	-2.08/-1.22/-2.00

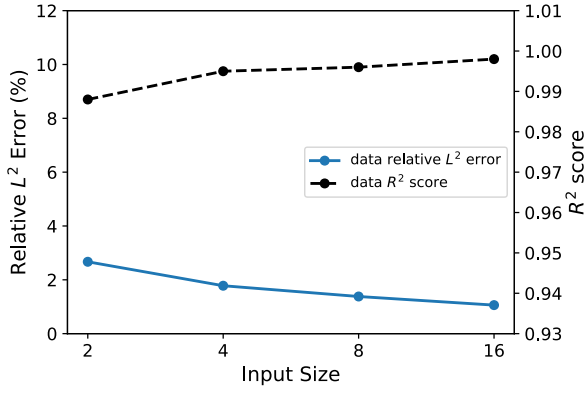


FIG. 4. Comparing the PROSE-PDE two-to-two model with varying amounts of input timestamps, i.e., input sizes and fixed output grid, i.e., $t > t_f/2$. The equations are in skeleton form and thus have unknown coefficients.

to extend the PROSE-PDE architecture to multidimensional nonlinear partial differential equations and to include non-time-dependent PDE.

ACKNOWLEDGMENTS

J.S., Y.L., and H.S. were supported in part by an AFOSR MURI FA9550-21-1-0084 and NSF DMS-2331033. Z.Z. was supported in part by NSF DMS-2331033.

J.S. and Y.L. conducted the research, contributed new analytic tools, analyzed the data, and wrote the paper. Z.Z. and H.S. designed the research, contributed new analytic tools, analyzed the data, and wrote the paper.

DATA AVAILABILITY

The datasets generated and analyzed during the current study, including materials characterization data and simulation results, are available online [76].

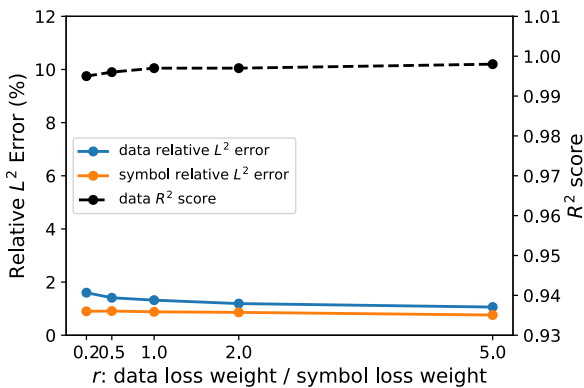


FIG. 5. Relative L^2 errors and R^2 scores with varying ratio of data and symbol loss weight. The results are reported using the model with the lowest symbol error in the validation set.

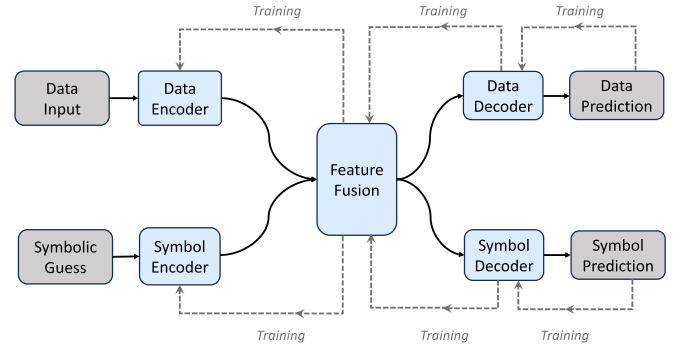


FIG. 6. Gradient information flow. We illustrate the gradient information flow for the PROSE-PDE model during the training phase. The training phase couples the modalities' parameters.

APPENDIX A: DATASET DETAILS

1. PDE types

The parameter-of-interest is listed for each type (see Table VI). Note that the training and testing data randomizes the values of the parameters.

a. Diffusion equation

The diffusion equation is

$$u_t - cu_{xx} = 0,$$

where $c = 3 \times 10^{-3}$, $t_{\text{final}} = 2$.

b. Porous medium equation

The porous medium equation is

$$u_t - (u^m)_{xx} = 0,$$

where $m = 2, 3, 4$ and $t_{\text{final}} = 0.1$.

c. Klein-Gordon equation

The Klein-Gordon equation is

$$u_{tt} - c^2 u_{xx} + m^2 c^4 u = 0,$$

TABLE VI. Choice of training and evaluation initial condition for different types of equations.

Equation type	Training initial condition	Testing initial condition
Heat	(A1): $n_{\text{max}} = 4, N = 2$	(A2): $\sigma = 1, l = 0.2$
Diff-react	(A1): $n_{\text{max}} = 4, N = 2$	(A2): $\sigma = 1, l = 0.2$
Klein-Gordon	(A1): $n_{\text{max}} = 4, N = 2$	(A2): $\sigma = 1, l = 0.2$
Sine-Gordon	(A1): $n_{\text{max}} = 4, N = 2$	(A2): $\sigma = 1, l = 0.2$
Cahn-Hilliard	(A1): $n_{\text{max}} = 4, N = 2$	(A2): $\sigma = 1, l = 0.2$
Viscous conservation	(A1): $n_{\text{max}} = 4, N = 2$	(A2): $\sigma = 1, l = 0.2$
Inviscid conservation	(A1): $n_{\text{max}} = 4, N = 2$	(A2): $\sigma = 1, l = 0.2$
KdV	(A1): $n_{\text{max}} = 2, N = 2$	(A4): $N = 2$
Advection	(A1): $n_{\text{max}} = 2, N = 2$	(A4): $N = 2$
Wave	(A1): $n_{\text{max}} = 2$	(A4): $N = 2$
Fokker-Planck	(A4): $N = 1$	(A5)
Porous medium	(A4): $N = 1$	(A6)

where $c = 1$, $m = 0.1$, and $t_{\text{final}} = 1$.

d. Sine-Gordon equation

The sine-Gordon equation is

$$u_{tt} - u_{xx} + c \sin(u) = 0,$$

where $c = 1$, and $t_{\text{final}} = 1$.

e. Cahn-Hilliard equation

The Cahn-Hilliard equation is

$$u_t + \epsilon^2 u_{xxxx} + 6(uu_x)_x = 0,$$

where $\epsilon = 0.01$, and $t_{\text{final}} = 0.5$.

f. Korteweg-de Vries (KdV) equation

The KdV equation is

$$u_t + \delta^2 u_{xxx} + uu_x = 0,$$

where $\delta = 0.022$, and $t_{\text{final}} = 1$.

g. Advection equation

The advection equation is

$$u_t + \beta u_x = 0,$$

where $\beta = 0.5$ and $t_{\text{final}} = 2$.

h. Wave equation

The wave equation is

$$u_{tt} - \beta^2 u_{xx} = 0,$$

where $\beta = 0.5$ and $t_{\text{final}} = 1$.

i. Diffusion reaction equation

The diffusion reaction equation is

$$u_t - \nu u_{xx} - \rho R(u) = 0,$$

where $\nu = 3 \times 10^{-3}$, $\rho = 1$ for $R = R_1, R_3, R_4$, $\rho = 0.1$ for R_2 , and $t_{\text{final}} = 2$, where

$$R_1(u) = u(1 - u),$$

$$R_2(u) = u,$$

$$R_3(u) = u^2(1 - u),$$

$$R_4(u) = u^2(1 - u)^2.$$

j. Viscous conservation law

The viscous conservation law is

$$u_t + kf(u)_x - \frac{\epsilon}{\pi} u_{xx} = 0,$$

where $k = 1$, $\epsilon = 0.01$, $t_{\text{final}} = 2$, and

$$f_1(u) = \frac{1}{2}u^2 \text{ (Burgers' equation),}$$

$$f_2(x) = u,$$

$$f_3(x) = \frac{1}{3}u^3,$$

$$f_4(x) = \sin(x).$$

k. Inviscid conservation law

The inviscid conservation law is

$$u_t + kf(u)_x = 0,$$

where $k = 1$, $t_{\text{final}} = 2$, and

$$f_1(u) = \frac{1}{2}u^2 \text{ (inviscid Burgers' equation),}$$

$$f_2(x) = \frac{1}{3}u^3,$$

$$f_3(x) = \sin(x).$$

l. Fokker-Planck equation

The Fokker-Planck equation is

$$u_t = Du_{xx} - \frac{D}{k_B T} [\nabla U(x)u]_x,$$

where $D = k_B T / \gamma$, where $k_B \approx 1.380649 \times 10^{-23}$ is the Boltzmann constant, $T = 300$ is absolute temperature, $\gamma = 6\pi\eta r$ represents the drag coefficient, $\eta = 10^{-3}$ is the fluid viscosity (randomized), and $r = 0.1 \times 10^{-6}$. $U(x) = c \cos(x/L)$, where $c = 5 \times 10^{-21}$, and $L = 0.1 \times 10^{-6}$. Set $t_{\text{final}} = 0.1$, and $x_{\text{final}} = 2 \times 10^{-6}$.

2. Initial conditions

We mainly consider periodic boundary conditions unless specified, and we use different types of initial conditions for different types of equations Table VI for the correspondence of PDEs and initial conditions.):

a. Superposition of sinusoidal waves

This is derived from PDEBench [77]:

$$u_0(x) = \sum_{k=k_1, \dots, k_N} A_i \sin(k_i x + \phi_i), \quad (\text{A1})$$

where $k_i = 2\pi n_i / L_x$, n_i is randomly selected integers from $[1, n_{\text{max}}]$, L_x is the spatial domain size. The amplitude A_i is random float uniformly chosen in $[0, 1]$, and ϕ_i is the randomly chosen phase in $(0, 2\pi)$. For all equations except advection and wave equation, after calculating (A1), we enforced absolute value with random signature and the window function with 10% probability.

b. Gaussian process

A Gaussian process is defined over the function $u_0(x)$ with zero mean and a covariance structure determined by the matrix K_x :

$$u_0(x) \sim \mathcal{N}(0, K_x), \quad (\text{A2})$$

where the covariance matrix K_x is obtained by the RBF kernel with $x_1 = x_2 = x$, and the RBF kernel is described as

$$k_{\text{RBF}}(x_1, x_2) = \sigma^2 \exp\left(-\frac{\|x_1 - x_2\|^2}{2l^2}\right). \quad (\text{A3})$$

c. Gaussian distribution

A Gaussian distribution is defined as:

$$u_0(x) = \sum_{i=1}^N A_i \exp\left(-\frac{|x - \mu_i|^2}{2\sigma_i^2}\right). \quad (\text{A4})$$

TABLE VII. Solvers for different types of equations.

Equation type	Generator
Heat	Method of Line
Klein-Gordon	Method of Line
Sine-Gordon	Method of Line
Porous medium	Method of Line
Cahn-Hilliard	Method of Line
Diff-react	PDEBench Ref. [77]
Viscous conservation	PDEBench Ref. [77]
Inviscid conservation	PDEBench Ref. [77]
Advection	Exact solution defined by IC
Wave	Exact solution defined by IC
KdV	Fourier spectral method Ref. [79]
Fokker-Planck	Matrix numerical method Ref. [78]

d. Uniform distribution

A uniform distribution is defined as:

$$u_0(x) \sim \text{Uniform}(x_l, x_r). \quad (\text{A5})$$

e. Quadratic function

A quadratic function is defined as:

$$u_0(x) = \max \left(-A \frac{(x - \mu)^2}{2\sigma^2} + A, 0 \right). \quad (\text{A6})$$

f. Periodization and normalization

We enforced periodicity for initial condition $u_0(x)$ by removing the linear function $l(x)$ that passes through the end-points of the domain, hence the modified initial condition is given by

$$u'_0(x) = u_0(x) - l(x). \quad (\text{A7})$$

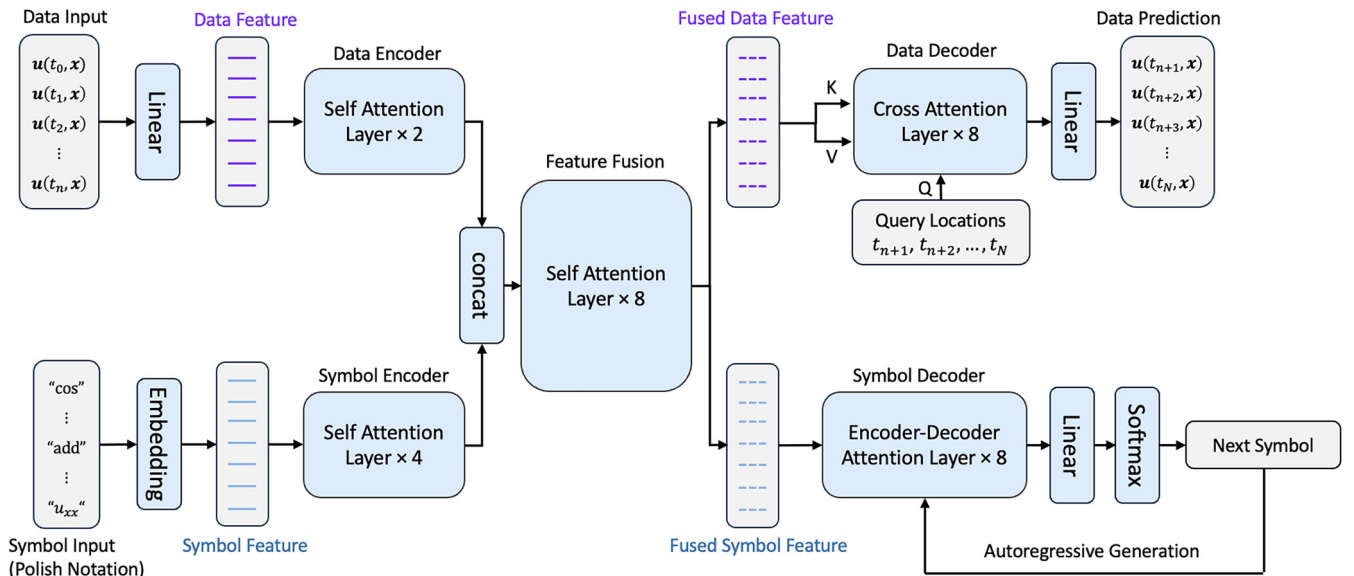


FIG. 7. PROSE-PDE architecture details. Data input and symbol input are embedded into data feature and symbol feature, respectively, before encoding and fusion through feature fusion. PROSE-PDE uses cross attention to construct the operator (upper-right structure) from fused data feature and evaluates it at query locations. PROSE-PDE generates symbolic expressions in the lower-right portion autoregressively. Attention blocks are displayed in Appendix C, where each layer also includes a feedforward network.

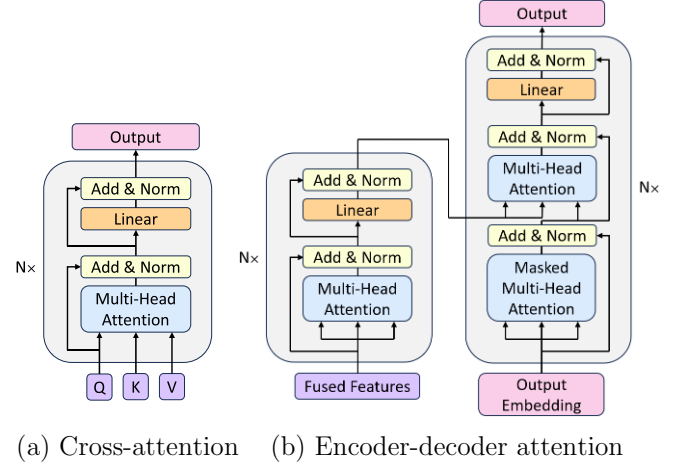


FIG. 8. Attention block details. Self-attention is a special case of cross attention with the same source.

We also normalize the initial condition in two distinct ways:

- (1) Adjust $u'_0(x)$ so that the range falls within $(0, u_{\max})$.
- (2) When the initial condition is represented by a probability distribution, we adjusted $u_0(x)$ such that the sum of probability is one.

3. Solvers

As detailed in Table VII, we use different solvers for different types of equations. For the diffusion-reaction equation and all types of conservation laws, we employ PDEBench [77]. The matrix numerical methods (MNMs) introduced in Ref. [78] are used for solving the Fokker-Planck equation, while the pseudospectral method from Ref. [79] is applied to the KdV equation. For advection and wave equations, we

TABLE VIII. Model hyperparameters. FFN means feedforward network.

Hidden dimension for attention	512	Hidden dimension for FFNs	2048
Number of attention heads	8	Fusion attention layers	8
Data encoder attention layers	2	Data decoder attention layers	8
Symbol encoder attention layers	4	Symbol decoder attention layers	8

utilize the exact solution defined by the initial conditions. The method of lines, which discretizes the PDE in space and solves the ODE in time, is used for the rest of the equations.

APPENDIX B: ARCHITECTURE DETAILS

Our network uses hierarchical attention for feature processing and fusion, and two transformer decoders for two downstream tasks. Figure 7 provides an overview of the architecture. The PROSE-PDE architecture contains five main components trained end-to-end: data encoder, symbol encoder, feature fusion, data decoder, and symbol decoder.

1. Encoders

Two separate transformer encoders are used to obtain domain-specific features. Given numerical data inputs and symbolic equation guesses (possibly empty or erroneous), the data encoder and symbol encoder first separately perform feature aggregation using self-attention. For a data input sequence $\mathbf{u}(t_0), \dots, \mathbf{u}(t_n)$, each element $\mathbf{u}(t_i)$, together with its time variable t_i , goes through a linear layer to form the data feature (purple feature sequence in Fig. 7). PROSE-PDE then uses self-attention to further process the data feature, where the time variables t_i serve as the positional encoding.

The symbolic input (in Polish notation) is a standard word sequence, which can be directly processed with self-attention layers. The word embedding (for operations, sign, mantissa, etc.) is randomly initialized and trainable. Sinusoidal positional encoding [58] is used for the symbol encoder.

2. Feature fusion

Hierarchical attention (multistream to one-stream) is used in this model for feature fusion. Separately processed data and symbol features are concatenated into a feature sequence, and further processed through self-attention layers where modality interaction occurs. Following Ref. [80], a learnable modality-type embedding is added to the fused features, signaling the source modality of each token. Positional encoding is not needed since it is already included in the individual encoders.

3. Data decoder

The data decoder constructs the operator via the cross-attention mechanism, establishing a link between the input-encoded time sequence (fused data features) and the output functions. The query locations, representing the independent variables of these output functions, serve as the evaluation points. Importantly, these query locations operate independently of each other, meaning that assessing the operator at one point, t_i , does not impact the evaluation of the operator at another point, t_j . As a result, the time and space complexity scales linearly with the number of query locations. In addition, since the evaluation points are independent of the network generation, this resembles the philosophy of the branch and trunk nets, see operator learning structure in Sec. III.

4. Symbol decoder

The symbol decoder is a standard encoder-decoder transformer, where the fused symbol feature is the context for generation. The output equation is produced using an autoregressive approach [58,71]: it starts with the start-of-sentence token and proceeds iteratively, generating each term of the equation based on prior predictions, until it encounters the end-of-sentence token for that specific equation. During evaluation time, greedy search (iterative selection of symbol with maximum probability) is used for efficient symbol generation. While beam search [81] can be used to improve the performance (e.g., percentage of valid expression outputs), we empirically find that greedy search is sufficient for obtaining valid mathematical expressions using the Polish notation formulation.

APPENDIX C: PRELIMINARY

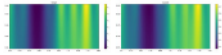
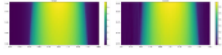
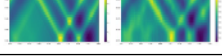
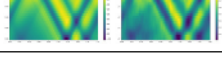
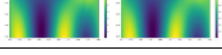
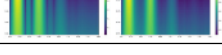
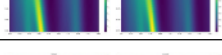
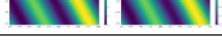
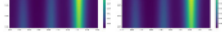
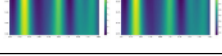
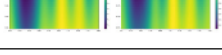
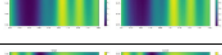
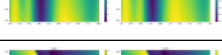
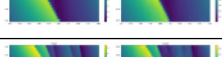
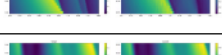
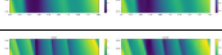
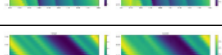
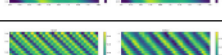
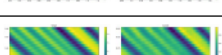
1. Transformers

A transformer operates on the principle of attention, adept at identifying long-range dependencies within data sources, as noted by Refs. [58–60]. It processes input by assigning varying degrees of importance to different segments of the data sequence, thereby focusing or “attending” to particular portions of the input for decision-making or output generation. The standard transformer model employs a self-attention

TABLE IX. Optimizer hyperparameters.

Learning rate	10^{-4}	Weight decay	10^{-4}
Scheduler	Cosine	Warmup steps	10% of total steps
Batch size per GPU	512	Gradient norm clip	1.0
Data loss weight α	5.0	Symbol loss weight β	1.0

TABLE X. Results for PROSE-PDE two-to-two model with skeleton inputs per type. The worst case prediction shows that the general trends and physical features are well captured by the model.

Type		Parameters	Metrics		Worst Prediction	
			Rel- L^2	R^2		
Diffusion $u_t = q \, u_{xx}, t_f = 2$		$q_c = 3 \times 10^{-3}$	0.40%	1.000		
Porous Medium $u_t = (u^m)_{xx}, t_f = 0.1$		$m \in (2, 3, 4)$	0.35%	0.995		
Klein-Gordon $u_{tt} + (q^{(2)})^2 (q^{(1)})^4 u = (q^{(1)})^2 u_{xx}, t_f = 1$		$q_c^{(1)} = 1$ $q_c^{(2)} = 0.1$	0.53%	1.000		
Sine-Gordon $u_{tt} + q \sin(u) = u_{xx}, t_f = 1$		$q_c = 1$	0.58%	1.000		
Wave $u_{tt} = q^2 u_{xx}, t_f = 1$		$q_c = 0.5$	0.33%	1.000		
Cahn-Hilliard $u_t + q^2 u_{xxxx} + c(u u_x)_x = 0, t_f = 0.5$		$q_c = 0.01$	0.38%	0.999		
Korteweg-De Vries $u_t + q^2 u_{xxx} + u u_x = 0, t_f = 1$		$q_c = 0.022$	0.94%	0.999		
Advection $u_t + q u_x = 0, t_f = 2$		$q_c = 0.5$	0.49%	1.000		
Fokker-Planck $u_t = D u_{xx} - \frac{1}{\gamma} (U(x)_x u)_x$ $t_f = 0.1, x_f = 2 \times 10^{-6}$	$U(x) = c \cos\left(\frac{x}{L}\right)$ $D = \frac{k_B T}{\gamma}$ $\gamma = 6\pi q^{(1)} r$	$c = 5 \times 10^{-21}$ $L = 10^{-7}$ $T = 300$ $r = 10^{-7}$ $q_c^{(1)} = 10^{-3}$	0.31%	0.998		
Diffusion Reaction $u_t = q^{(1)} u_{xx} + q^{(2)} R(u)$ $t_f = 2$	$R(u) = u$	$q_c^{(1)} = 3 \times 10^{-3}$ $q_c^{(2)} = 0.1$	0.44%	1.000		
	$R(u) = u(1 - u)$	$q_c^{(1)} = 3 \times 10^{-3}$ $q_c^{(2)} = 1$	0.45%	0.999		
	$R(u) = u^2(1 - u)$		0.54%	1.000		
	$R(u) = u^2(1 - u)^2$		0.45%	1.000		
Conservation Law $u_t = -q^{(1)}(f(u))_x + \frac{q^{(2)}}{\pi} u_{xx}$ $t_f = 2$	Burgers': $f(u) = \frac{1}{2} u^2$	$q_c^{(1)} = 1$ $q_c^{(2)} = 0.01$	1.16%	0.999		
		$q_c^{(1)} = 1$ $q_c^{(2)} = 0$	2.45%	0.993		
	$f(u) = \frac{1}{3} u^3$	$q_c^{(1)} = 1$ $q_c^{(2)} = 0.01$	0.90%	0.999		
		$q_c^{(1)} = 1$ $q_c^{(2)} = 0$	2.19%	0.994		
	$f(u) = \sin(u)$	$q_c^{(1)} = 1$ $q_c^{(2)} = 0.01$	1.91%	0.996		
		$q_c^{(1)} = 1$ $q_c^{(2)} = 0$	4.26%	0.987		
	$f(u) = u$	$q_c^{(1)} = 1$ $q_c^{(2)} = 0.01$	2.14%	0.997		

framework, as described by Refs. [61,62], which allows it to discern complex patterns in extensive time-series data.

Specifically, let us denote the input time series data as $X \in \mathbb{R}^{n \times d}$, where n is the number of time steps and d is the di-

mension of each element in the time series. Self-attention first computes the projections: query $Q = XW^Q$, key $K = XW^K$ and value $V = XW^V$, where $W^Q \in \mathbb{R}^{d \times d_k}$, $W^K \in \mathbb{R}^{d \times d_k}$, and $W^V \in \mathbb{R}^{d \times d_v}$. It then outputs the context $C \in \mathbb{R}^{n \times d_v}$

via

$$C = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \quad (\text{C1})$$

where the softmax function is calculated over all entries of each row. Self-attention discovers relationships among various elements within a time sequence. Predictions often depend on multiple data sources, making it crucial to understand the interactions and encode various time series data (see Sec. II B for details). The self-attention is also used in the development of the cross-attention mechanism [50,53,82]. Given two input time series data X, Y , cross attention computes the query, key, and value as $Q = XW^Q$, $K = YW^K$, and $V = YW^V$. In the case where Y represents the output of a decoder and X represents the output of an encoder, the cross attention, which directs its focus from X to Y , is commonly referred to as encoder-decoder attention [58]. Encoder-decoder attention serves as a crucial component within autoregressive models [53,58,83]. The autoregressive model operates by making predictions for a time series iteratively, one step at a time. To achieve this, it utilizes the previous step's generated output as additional input for the subsequent prediction. This approach has demonstrated the capacity for mitigating accumulation of errors [71], which makes it desirable for longer-time predictions.

Figure 8 contains the transformer architecture details.

APPENDIX D: EXPERIMENT SETUP

1. Training

A standard cross-entropy loss $\mathcal{L}_s$ is used for the symbolic outputs. Mean and standard deviation of the input sequence are computed, which are then used to normalize both the inputs and the labels. Mean squared error (in the normalized space) $\mathcal{L}_d$ is used for the data predictions.

The data loss $\mathcal{L}_d$ and symbol loss $\mathcal{L}_s$ are combined to form the final loss function $\mathcal{L} = \alpha\mathcal{L}_d + \beta\mathcal{L}_s$, where the weights α, β are hyperparameters. Unless otherwise specified, the models are trained using the AdamW optimizer for 30 epochs where each epoch is 2000 steps. On a single NVIDIA GeForce RTX 4090 GPUs with 24 GB memory each, the training takes about 4.5 hours.

2. Hyperparameters

The PROSE model hyperparameters are summarized in Table VIII, and the optimizer hyperparameters are summarized in Table IX.

For the FNO model in Sec. V B, we use four layers of standard two-dimensional (2d) FNO to process the input data. The number of modes to keep in each dimension is set to 16, and the number of hidden channels is set to 64.

For the DeepONet model in Sec. V B, we employ the unstacked DeepONet architecture, consisting of a single trunk network and a single branch network. The input vectors are then passed through the branch network, producing an output with a basis dimension of $p \times \text{dim}_{\text{output}} = 20 \times 128$. Simultaneously, the query point is processed through the trunk network, which also outputs a vector with the same dimension p . Each element of output solution at the query point is obtained by taking the inner product of the outputs from the trunk net and the corresponding element of outputs from the branch net.

APPENDIX E: ADDITIONAL RESULTS

We present additional results in this section. Table X contains the errors per equation type as well as sample outputs.

-
- [1] R. Bommasani, D. A. Hudson, E. Adeli, R. Altman, S. Arora, S. von Arx, M. S. Bernstein, J. Bohg, A. Bosselut, E. Brunskill *et al.*, On the opportunities and risks of foundation models, [arXiv:2108.07258](https://arxiv.org/abs/2108.07258).
 - [2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, Bert: Pre-training of deep bidirectional transformers for language understanding, [arXiv:1810.04805](https://arxiv.org/abs/1810.04805).
 - [3] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever *et al.*, Improving language understanding by generative pre-training (2018).
 - [4] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, Language models are unsupervised multitask learners (2019), https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.
 - [5] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, Language models are few-shot learners, in *Advances in Neural Information Processing Systems*, edited by H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Vol. 33 (Curran Associates, Inc., 2020), pp. 1877–1901.
 - [6] A. Ramesh, M. Pavlov, G. Goh, S. Gray, C. Voss, A. Radford, M. Chen, and I. Sutskever, Zero-shot text-to-image generation, in *International Conference on Machine Learning* (PMLR, 2021), pp. 8821–8831.
 - [7] A. Ramesh, P. Dhariwal, A. Nichol, C. Chu, and M. Chen, Hierarchical text-conditional image generation with clip latents, [arXiv:2204.06125](https://arxiv.org/abs/2204.06125).
 - [8] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, High-resolution image synthesis with latent diffusion models, in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (IEEE, Piscataway, NJ, 2022), pp. 10684–10695.
 - [9] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, Llama: Open and efficient foundation language models, [arXiv:2302.13971](https://arxiv.org/abs/2302.13971).
 - [10] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, Llama 2: Open foundation and fine-tuned chat models, [arXiv:2307.09288](https://arxiv.org/abs/2307.09288).

- [11] Y. Efendiev, W. T. Leung, G. Lin, and Z. Zhang, Efficient hybrid explicit-implicit learning for multiscale problems, *J. Comput. Phys.* **467**, 111326 (2022).
- [12] K. Champion, B. Lusch, J. N. Kutz, and S. L. Brunton, Data-driven discovery of coordinates and governing equations, *Proc. Natl. Acad. Sci. USA* **116**, 22445 (2019).
- [13] S. Imani, L. Du, and H. Shrivastava, Mathprompter: Mathematical reasoning using large language models, [arXiv:2303.05398](https://arxiv.org/abs/2303.05398).
- [14] Y. Zhuang, Y. Yu, K. Wang, H. Sun, and C. Zhang, Toolqa: A dataset for LLM question answering with external tools, in *Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS '23* (Curran Associates Inc., Red Hook, NY, 2023), pp. 50117–50143.
- [15] M. Turpin, J. Michael, E. Perez, and S. Bowman, Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting, *Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS '23* (Curran Associates Inc., Red Hook, NY, 2023), pp. 74952–74965.
- [16] T. Chen and H. Chen, Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems, *IEEE Trans. Neural Netw.* **6**, 911 (1995).
- [17] T. Chen and H. Chen, Approximations of continuous functionals by neural networks with application to dynamic systems, *IEEE Trans. Neural Netw.* **4**, 910 (1993).
- [18] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis, Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators, *Nat. Mach. Intell.* **3**, 218 (2021).
- [19] L. Lu, X. Meng, S. Cai, Z. Mao, S. Goswami, Z. Zhang, and G. E. Karniadakis, A comprehensive and fair comparison of two neural operators (with practical extensions) based on fair data, *Comput. Methods Appl. Mech. Eng.* **393**, 114778 (2022).
- [20] P. Jin, S. Meng, and L. Lu, Mionet: Learning multiple-input operators via tensor product, *SIAM J. Sci. Comput.* **44**, A3490 (2022).
- [21] G. Lin, C. Moya, and Z. Zhang, B-DeepONet: An enhanced Bayesian DeepONet for solving noisy parametric PDEs using accelerated replica exchange SGLD, *J. Comput. Phys.* **473**, 111713 (2023).
- [22] G. Lin, C. Moya, and Z. Zhang, Learning the dynamical response of nonlinear non-autonomous dynamical systems with deep operator neural networks, *Eng. Appl. Artif. Intell.* **125**, 106689 (2023).
- [23] Z. Zhang, W. T. Leung, and H. Schaeffer, BelNet: Basis enhanced learning, a mesh-free neural operator, *Proc. R. Soc. London, Ser. A* **479**, 20230043 (2023).
- [24] Z. Zhang, W. T. Leung, and H. Schaeffer, A discretization-invariant extension and analysis of some deep operator networks, *J. Comput. Applied Math.* **456**, 116226 (2025).
- [25] Z. Zhang, C. Moya, L. Lu, G. Lin, and H. Schaeffer, D2NO: Efficient handling of heterogeneous input function spaces with distributed deep neural operators, *Comput. Methods Appl. Mech. Eng.* **428**, 117084 (2024).
- [26] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar, Fourier neural operator for parametric partial differential equations, [arXiv:2010.08895](https://arxiv.org/abs/2010.08895).
- [27] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar, Neural operator: Graph Kernel network for partial differential equations, [arXiv:2003.03485](https://arxiv.org/abs/2003.03485).
- [28] G. Wen, Z. Li, K. Azizzadenesheli, A. Anandkumar, and S. M. Benson, U-FNO—An enhanced Fourier neural operator-based deep-learning model for multiphase flow, *Adv. Water Resour.* **163**, 104180 (2022).
- [29] Z. Li, D. Z. Huang, B. Liu, and A. Anandkumar, Fourier neural operator with learned deformations for PDEs on general geometries, *J. Mach. Learn. Research* **24**, 18593 (2023).
- [30] J. Pathak, S. Subramanian, P. Harrington, S. Raja, A. Chattopadhyay, M. Mardani, T. Kurth, D. Hall, Z. Li, K. Azizzadenesheli *et al.*, FourCastNet: A global data-driven high-resolution weather model using adaptive fourier neural operators, [arXiv:2202.11214](https://arxiv.org/abs/2202.11214).
- [31] Z. Jiang, M. Zhu, and L. Lu, Fourier-Mionet: Fourier-enhanced multiple-input neural operators for multiphase modeling of geological carbon sequestration, *Reliab. Eng. Syst. Saf.* **251**, 110392 (2024).
- [32] M. Zhu, S. Feng, Y. Lin, and L. Lu, Fourier-DeepONet: Fourier-enhanced deep operator networks for full waveform inversion with improved accuracy, generalizability, and robustness, *Comput. Methods Appl. Mech. Eng.* **416**, 116300 (2023).
- [33] Z. Mao, L. Lu, O. Marxen, T. A. Zaki, and G. E. Karniadakis, DeepM&Mnet for hypersonics: Predicting the coupled flow and finite-rate chemistry behind a normal shock using neural-network approximation of operators, *J. Comput. Phys.* **447**, 110698 (2021).
- [34] P. C. Di Leoni, L. Lu, C. Meneveau, G. E. Karniadakis, and T. A. Zaki, Neural operator prediction of linear instability waves in high-speed boundary layers, *J. Comput. Phys.* **474**, 111793 (2023).
- [35] S. Mao, R. Dong, L. Lu, K. M. Yi, S. Wang, and P. Perdikaris, Ppdonet: Deep operator networks for fast prediction of steady-state solutions in disk–planet systems, *Astrophys. J. Lett.* **950**, L12 (2023).
- [36] M. Yin, N. Charon, R. Brody, L. Lu, N. Trayanova, and M. Maggioni, Dimon: Learning solution operators of partial differential equations on a diffeomorphic family of domains, [arXiv:2402.07250](https://arxiv.org/abs/2402.07250).
- [37] C. Moya, A. Mollaali, Z. Zhang, L. Lu, and G. Lin, Conformalized-deeponet: A distribution-free framework for uncertainty quantification in deep operator networks, *Physica D: Nonlinear Phenomena* **471**, 134418 (2025).
- [38] L. Lu, R. Pestourie, S. G. Johnson, and G. Romano, Multifidelity deep neural operators for efficient learning of partial differential equations with application to fast inverse design of nanoscale heat transport, *Phys. Rev. Res.* **4**, 023210 (2022).
- [39] Z. Zhang, C. Moya, W. T. Leung, G. Lin, and H. Schaeffer, Bayesian deep operator learning for homogenized to fine-scale maps for multiscale PDE, *Multiscale Model. Simul.* **22**, 956 (2024).
- [40] A. A. Howard, M. Perego, G. E. Karniadakis, and P. Stinis, Multifidelity deep operator networks for data-driven and physics-informed problems, *J. Comput. Phys.* **493**, 112462 (2023).
- [41] W. T. Leung, G. Lin, and Z. Zhang, NH-PINN: Neural homogenization-based physics-informed neural network for multiscale problems, *J. Comput. Phys.* **470**, 111539 (2022).

- [42] Y. Liu, Z. Zhang, and H. Schaeffer, PROSE: Predicting operators and symbolic expressions using multimodal transformers, *Neural Networks* **180**, 106707 (2024).
- [43] L. Yang, S. Liu, T. Meng, and S. J. Osher, In-context operator learning with data prompts for differential equation problems, *Proc. Natl. Acad. Sci. USA* **120**, e2310142120 (2023).
- [44] L. Yang, T. Meng, S. Liu, and S. J. Osher, Prompting in-context operator learning with sensor data, equations, and natural language, [arXiv:2308.05061](#).
- [45] L. Yang and S. J. Osher, PDE generalization of in-context operator networks: A study on 1D scalar nonlinear conservation laws, *J. Comput. Phys.* **519**, 113379 (2024).
- [46] Z. Ye, X. Huang, L. Chen, H. Liu, Z. Wang, and B. Dong, Pdeformer: Towards a foundation model for one-dimensional partial differential equations, [arXiv:2402.12652](#).
- [47] J. Shen, T. Marwah, and A. Talwalkar, Ups: Towards foundation models for PDE solving via cross-modal adaptation, [arXiv:2403.07187](#).
- [48] M. McCabe, B. R.-S. Blancard, L. H. Parker, R. Ohana, M. Cranmer, A. Bietti, M. Eickenberg, S. Golkar, G. Krawezik, F. Lanusse *et al.*, Multiple physics pretraining for physical surrogate models, [arXiv:2310.02994](#).
- [49] Z. Zhang, MODNO: Multi operator learning with distributed neural operators, *Comput. Methods. Appl. Mech. Eng.* **431**, 117229 (2024).
- [50] J. Lu, D. Batra, D. Parikh, and S. Lee, Vilbert: pretraining task-agnostic visiolinguistic representations for vision-and-language tasks, in *Proceedings of the 33rd International Conference on Neural Information Processing Systems* (Curran Associates Inc., Red Hook, NY, 2019), pp. 13–23.
- [51] C. Sun, A. Myers, C. Vondrick, K. Murphy, and C. Schmid, Videobert: A joint model for video and language representation learning, in *Proceedings of the IEEE/CVF International Conference on Computer Vision* (IEEE, Piscataway, NJ, 2019), pp. 7464–7473.
- [52] H. Tan and M. Bansal, Lxmert: Learning cross-modality encoder representations from transformers, [arXiv:1908.07490](#).
- [53] R. Li, S. Yang, D. A. Ross, and A. Kanazawa, Ai choreographer: Music conditioned 3D dance generation with aist++, in *Proceedings of the IEEE/CVF International Conference on Computer Vision* (IEEE, Piscataway, NJ, 2021), pp. 13401–13412.
- [54] P. Xu, X. Zhu, and D. A. Clifton, Multimodal learning with transformers: A survey, *IEEE Trans. Pattern Anal. Mach. Intell.* **45**, 12113 (2023).
- [55] D. Feng, C. Haase-Schütz, L. Rosenbaum, H. Hertlein, C. Glaeser, F. Timm, W. Wiesbeck, and K. Dietmayer, Deep multi-modal object detection and semantic segmentation for autonomous driving: Datasets, methods, and challenges, *IEEE Trans. Intell. Transport. Syst.* **22**, 1341 (2020).
- [56] Y. Liu, J. Zhang, L. Fang, Q. Jiang, and B. Zhou, Multimodal motion prediction with stacked transformers, in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (IEEE, Piscataway, NJ, 2021), pp. 7577–7586.
- [57] L. H. Li, M. Yatskar, D. Yin, C.-J. Hsieh, and K.-W. Chang, Visualbert: A simple and performant baseline for vision and language, [arXiv:1908.03557](#).
- [58] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, Attention is all you need, in *Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS '17* (Curran Associates Inc., Red Hook, NY, 2017), pp. 6000–6010.
- [59] Z. Dai, Z. Yang, Y. Yang, J. Carbonell, Q. V. Le, and R. Salakhutdinov, Transformer-xl: Attentive language models beyond a fixed-length context, [arXiv:1901.02860](#).
- [60] I. Beltagy, M. E. Peters, and A. Cohan, Longformer: The long-document transformer, [arXiv:2004.05150](#).
- [61] D. Bahdanau, K. Cho, and Y. Bengio, Neural machine translation by jointly learning to align and translate, [arXiv:1409.0473](#).
- [62] K. Xu, J. Ba, R. Kiros, K. Cho, A. Courville, R. Salakhutdinov, R. Zemel, and Y. Bengio, Show, attend and tell: Neural image caption generation with visual attention, in *International Conference on Machine Learning* (PMLR, 2015), pp. 2048–2057.
- [63] S. Liang and H. Yang, Finite expression method for solving high-dimensional partial differential equations, [arXiv:2206.10121](#).
- [64] Z. Jiang, C. Wang, and H. Yang, Finite expression methods for discovering physical laws from data, [arXiv:2305.08342](#).
- [65] H. A. Pogorzelski, J. Lukasiewicz, J. Slupecki, and P. Wydawnictwo, Remarks on Nicod's axiom and on "generalizing deduction," *J. Symb. Log.* **30**, 376 (1965).
- [66] F. Charton, Linear algebra with transformers, [arXiv:2112.01898](#).
- [67] S. d'Ascoli, P.-A. Kamienny, G. Lample, and F. Charton, Deep symbolic regression for recurrent sequences, [arXiv:2201.04600](#).
- [68] P.-A. Kamienny, S. d'Ascoli, G. Lample, and F. Charton, End-to-end symbolic regression with transformers, in *Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS '22* (Curran Associates Inc., Red Hook, NY, 2022).
- [69] K. S. Tai, R. Socher, and C. D. Manning, Improved semantic representations from tree-structured long short-term memory networks, in *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)* (Association for Computational Linguistics, 2015), pp. 1556–1566.
- [70] C. Dyer, A. Kuncoro, M. Ballesteros, and N. A. Smith, Recurrent neural network grammars, in *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (Association for Computational Linguistics, 2016), pp. 199–209.
- [71] L. Floridi and M. Chiriatti, GPT-3: Its nature, scope, limits, and consequences, *Minds Mach.* **30**, 681 (2020).
- [72] S.-M. Udrescu and M. Tegmark, Ai Feynman: A physics-inspired method for symbolic regression, *Sci. Adv.* **6**, eaay2631 (2020).
- [73] H. Schaeffer, Learning partial differential equations via data discovery and sparse optimization, *Proc. R. Soc. London, Ser. A* **473**, 20160446 (2017).
- [74] H. Schaeffer and S. G. McCalla, Sparse model selection via integral terms, *Phys. Rev. E* **96**, 023302 (2017).

- [75] H. Schaeffer, G. Tran, and R. Ward, Extracting sparse high-dimensional dynamics from limited data, *SIAM J. Appl. Math.* **78**, 3279 (2018).
- [76] Y. Liu, J. Sun, Z. Zhang, and H. Schaeffer, PROSE: Predicting multiple operators and symbolic expressions (2023), <https://github.com/felix-lyx/prose>.
- [77] M. Takamoto, T. Praditia, R. Leiteritz, D. MacKinlay, F. Alesiani, D. Pflüger, and M. Niepert, Pdebench: An extensive benchmark for scientific machine learning, in *Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS '22* (Curran Associates Inc., Red Hook, NY, 2022).
- [78] V. Holubec, K. Kroy, and S. Steffenoni, Physically consistent numerical solver for time-dependent Fokker-Planck equations, *Phys. Rev. E* **99**, 032117 (2019).
- [79] N. J. Zabusky and M. D. Kruskal, Interaction of “solitons” in a collisionless plasma and the recurrence of initial states, *Phys. Rev. Lett.* **15**, 240 (1965).
- [80] W. Kim, B. Son, and I. Kim, Vilt: Vision-and-language transformer without convolution or region supervision, in *International Conference on Machine Learning* (PMLR, 2021), pp. 5583–5594.
- [81] Y. Wu, M. Schuster, Z. Chen, Q. V. Le, M. Norouzi, W. Macherey, M. Krikun, Y. Cao, Q. Gao, K. Macherey, J. Klingner, A. Shah, M. Johnson, X. Liu, L. Kaiser, S. Gouws, Y. Kato, T. Kudo, H. Kazawa, K. Stevens *et al.*, Google’s neural machine translation system: Bridging the gap between human and machine translation, [arXiv:1609.08144](https://arxiv.org/abs/1609.08144).
- [82] Y.-H. H. Tsai, S. Bai, P. P. Liang, J. Z. Kolter, L.-P. Morency, and R. Salakhutdinov, Multimodal transformer for unaligned multimodal language sequences, in *Proceedings of the Conference. Association for Computational Linguistics. Meeting* (NIH Public Access, 2019), Vol. 2019, p. 6558.
- [83] A. Graves, Generating sequences with recurrent neural networks, [arXiv:1308.0850](https://arxiv.org/abs/1308.0850).